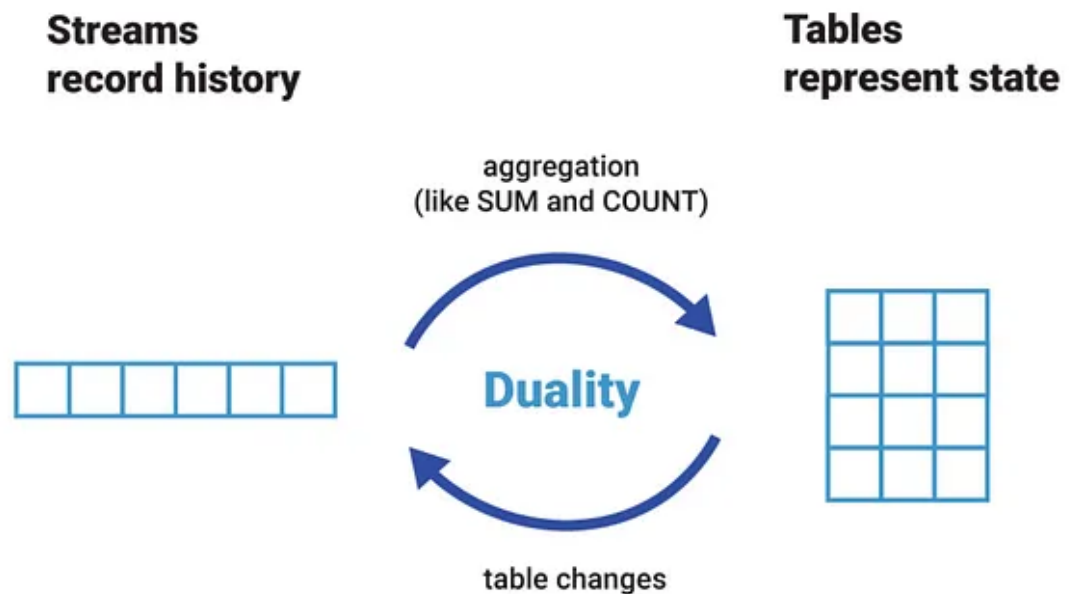


KStream vs KTable

[Kamini Kamal](#)



KStream and KTable are two important concepts in Apache Kafka's Streams library, which is a powerful framework for building real-time stream processing applications. They represent different types of data abstractions and offer different semantics for processing data streams. Here are the main differences between KStream and KTable:

1. Data Semantics:

- **KStream:** It represents an unbounded, continuously updating stream of records. Each record in a KStream consists of a key, a value, and a timestamp. KStream can have multiple records with the same key, and new records can be continuously appended to the stream.
- **KTable:** It represents a changelog stream of records that are organized as a table. Each record in a KTable has a unique key and a value. Unlike KStream, KTable maintains the latest value for each key and updates the table with new records. It can be

seen as a materialized view of the underlying stream.

2. Mutability:

- **KStream:** It is mutable, meaning you can modify and update the stream as new records arrive. You can perform various operations on KStream, such as filtering, mapping, aggregating, and joining. KStream allows stateful operations, as it retains the entire history of records and can be updated based on new incoming data.
- **KTable:** It is immutable, meaning once a record is written to a KTable, it cannot be modified. Instead, new records with the same key will update the table, effectively replacing the previous value. KTable operations are generally stateless and do not require maintaining a history of all records.

3. Querying:

- **KStream:** It is suitable for continuous querying and processing of streaming data. You can perform stateful operations and compute aggregations over time windows. KStream allows you to transform and manipulate the stream data directly as it arrives.
- **KTable:** It is suitable for point lookups and joins. KTable maintains the latest value for each key, making it efficient for performing queries and point reads based on the key. It is especially useful for maintaining materialized views and building interactive applications.

4. Use Cases:

- **KStream:** It is commonly used for real-time analytics, event-driven architectures, stream transformations, and complex stream processing scenarios. It is ideal when you need to process an unbounded stream of records and continuously update the stream.

- **KTable:** It is commonly used for building materialized views, performing table lookups, maintaining state, and performing joins with other KTables or KStreams. It is suitable when you need a snapshot view of the stream's latest values for each key.

In summary, KStream represents an unbounded stream of records that can be continuously updated and processed, while KTable represents a changelog stream organized as a table with the latest values for each key. The choice between KStream and KTable depends on the nature of your data, the processing requirements, and the type of queries and operations you need to perform.

When to use KStream vs KTable?

The choice between using KStream and KTable in your stream processing application depends on the specific requirements of your use case and the type of operations you need to perform. Here are some scenarios where you might choose to use KStream or KTable:

Use KStream when:

1. **Continuous Stream Processing:** If you have a continuous stream of records where new data is continuously appended, and you need to perform transformations, filtering, aggregations, or complex stream processing operations, KStream is a suitable choice. KStream allows you to process the data as it arrives and maintain stateful operations.
2. **Real-time Analytics:** If your use case involves real-time analytics, where you need to compute aggregations over time windows, perform sliding window operations, or derive insights from the stream of records, KStream is a good fit. It enables you to perform various analytical operations on the stream data.
3. **Complex Stream Transformations:** When you require complex transformations and operations on the stream data, such as mapping, filtering, branching, or joining multiple streams

together, KStream provides the necessary flexibility and functionality. It allows you to manipulate and transform the stream records based on your application logic.

Use KTable when:

1. **Materialized Views:** If you need to maintain the latest state of the stream data for each key and perform point lookups efficiently, KTable is a suitable choice. KTable can be used to build materialized views of the stream, where you can retrieve the current value for a specific key quickly.
2. **Table Joins:** If your use case involves joining the stream data with other tables or streams based on a common key, KTable provides a convenient way to perform these joins. It allows you to join KTables or KStreams together efficiently based on their keys.
3. **Interactive Queries:** When you need to support interactive queries and retrieve the latest value for a specific key in real-time, KTable is a good option. KTable maintains the latest value for each key, making it efficient for point lookups and queries.
4. **Stateful Processing:** If your application requires maintaining state and processing records based on the complete history of the stream data, KTable can be used to maintain the latest state for each key. It allows you to update and process the table based on new incoming records.

In many cases, you might find that a combination of KStream and KTable is needed to address different aspects of your stream processing requirements. Kafka Streams provides APIs and operators to seamlessly integrate both types and perform operations across them.

Result of KTable and KStream joins

When performing a join operation between a KTable and a KStream

in Apache Kafka's Streams library, the result is typically a new KStream. The join operation combines records from the KTable and the KStream based on a common key and produces an output stream with the joined records. The result depends on the type of join operation performed:

1. **Inner Join:** An inner join between a KTable and a KStream will produce a new KStream that includes only the records where there is a match between the key of the KTable and the key of the KStream. The result will contain the joined records, combining the values from both the KTable and the KStream.
2. **Left Join:** A left join between a KTable and a KStream will produce a new KStream that includes all records from the KStream, along with matching records from the KTable. If there is no matching record in the KTable for a particular key in the KStream, a null value will be included in the result.
3. **Outer Join:** An outer join between a KTable and a KStream will produce a new KStream that includes all records from both the KTable and the KStream, whether or not there is a match between their keys. If there is no matching record in either the KTable or the KStream for a particular key, a null value will be included in the result.

The resulting KStream from the join operation can be further processed, filtered, transformed, or aggregated using the available operators and functions in Kafka Streams. It allows you to perform various operations on the joined stream to derive insights or produce the desired output based on your application requirements.

Why is Kstream given priority in joins over KTable?

In Apache Kafka's Streams library, when performing a join operation between a KStream and a KTable, the KStream is typically given priority in the join. This means that the result of the join operation will be a new KStream with the semantics and characteristics of the KStream.

The reason behind this priority is related to the continuous and potentially unbounded nature of a KStream. KStreams represent streams of records that are continuously updating, with new records being appended as they arrive. On the other hand, KTables represent changelog streams organized as tables, which maintain the latest value for each key.

Given the dynamic nature of KStreams, they are often used for real-time processing, complex transformations, and continuous stream operations. They can have multiple records with the same key and can perform stateful operations, retaining the entire history of records. This makes KStreams more suitable for stream processing scenarios that require continuous updates and transformations.

By giving priority to KStream in joins, it ensures that the resulting joined stream maintains the characteristics of a KStream. The joined stream can be further processed, transformed, and updated as new records arrive. It allows you to leverage the full power of stream processing operations on the joined data, such as filtering, mapping, aggregating, and windowing, which are well-supported in the KStream API.

However, it's important to note that Kafka Streams provides flexibility and allows you to perform joins between KStreams and KTables in different ways. You can use the `leftJoin()` or `outerJoin()` methods to explicitly perform left or outer joins, where the result retains both the characteristics of the KStream and the KTable. This flexibility enables you to choose the appropriate join type based on your specific use case and requirements.

References

[**Streams and Tables in Apache Kafka: A Primer**](#)

[**This four-part series explores the core**](#)

fundamentals of Kafka's storage and processing layers and how they interrelate...

<https://kafka.apache.org/0102/documentation/streams/developer-guide>